

Algoritmica grafurilor - Cursul 6

6 novembre 2020

- 1 **Problema arborelui parțial de cost minim**
 - Metoda generală MST
 - Algoritmul lui Prim
 - Algoritmul lui Kruskal
- 2 **Cuplaje**
 - Cuplaje maxime – Acoperire minimă cu muchii
- 3 **Exerciții pentru seminarul din săptămâna următoare**
- 4 **Exerciții rezolvate (parțial)**

Problema arborelui parțial de cost minim

Problema MST. Dat $G = (V, E)$ un graf și $c : E \rightarrow \mathbb{R}$ ($c(e)$ este costul muchiei e) găsiți $T^* \in \mathcal{T}_G$ astfel încât

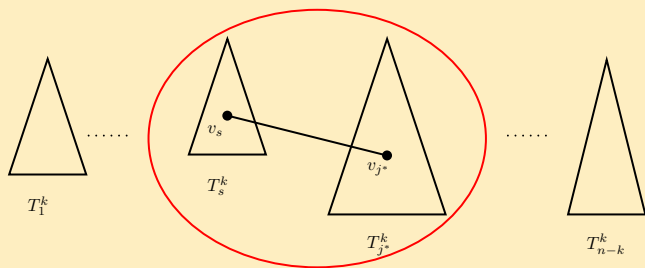
$$c(T^*) = \min_{T \in \mathcal{T}_G} c(T),$$

unde $c(T) = \sum_{e \in E(T)} c(e)$.

- Se pornește cu familia $\mathcal{T}^0 = (T_1^0, T_2^0, \dots, T_n^0)$ de arbori disjuncți:
 $T_i^0 = (\{i\}, \emptyset)$, $i = \overline{1, n}$.
- La fiecare pas k ($0 \leq k \leq n - 2$), din familia $\mathcal{T}^k = (T_1^k, T_2^k, \dots, T_{n-k}^k)$ de $n - k$ arbori disjuncți astfel încât $V = \bigcup_{i=1}^{n-k} V(T_i^k)$ și $\bigcup_{i=1}^{n-k} E(T_i^k) \subseteq E$, construiește $\mathcal{T}^{k+1}$ după cum urmează:
 - alege $T_s^k \in \mathcal{T}^k$;
 - determină o muchie de cost minim $e^* = v_s v_{j^*}$ din mulțimea de muchii ale lui G cu o extremitate $v_s \in V(T_s^k)$ și cealaltă în $V \setminus V(T_s^k)$ ($v_{j^*} \in V(T_{j^*}^k)$);
 - $\mathcal{T}^{k+1} = (\mathcal{T}^k \setminus \{T_s^k, T_{j^*}^k\}) \cup T$, unde T este arborele obținut din T_s^k și $T_{j^*}^k$ prin adăugarea muchiei e^* .

Remarci

- Observăm că, dacă, la un anumit pas, nu există nicio muchie cu o extremitate în $V(T_s^k)$ și cealaltă în $V \setminus V(T_s^k)$, atunci G nu este conex și nu există vreun MST în G .
- Construcția de mai sus este sugerată în imaginea de mai jos:



- Familia $\mathcal{T}^{n-1}$ are doar un arbore, T_1^{n-1} .

Teorema 1

Dacă $G = (V, E)$ este un graf conex cu $V = \{1, 2, \dots, n\}$, atunci T_1^{n-1} construit de algoritmul anterior este un MST al lui G .

Demonstrație: Demonstrăm (prin inducție) că (*) $\forall k \in \{0, \dots, n-1\}$ există un arbore parțial T_k^* , MST al lui G , astfel încât

$$E(\mathcal{T}^k) = \bigcup_{i=1}^{n-k} E(T_i^k) \subseteq E(T_k^*).$$

În particular, pentru $k = n-1$, $E(\mathcal{T}^{n-1}) = E(T_1^{n-1}) \subseteq E(T_{n-1}^*)$ implică $T_1^{n-1} = T_{n-1}^*$ și teorema este demonstrată.

Pentru $k = 0$, avem $E(\mathcal{T}^0) = \emptyset$ și, deoarece G este conex, există un MST T_0^* ; astfel, proprietatea (*) este adevărată.

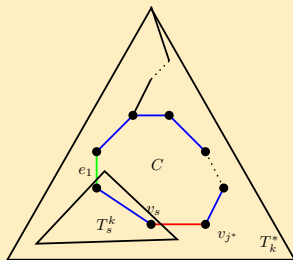
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Demonstrație (continuare). Dacă proprietatea (*) are loc pentru $0 \leq k \leq n - 2$, atunci există un MST al lui G , T_k^* , astfel încât $E(\mathcal{T}^k) \subseteq E(T_k^*)$. Din construcție, $E(\mathcal{T}^{k+1}) = E(\mathcal{T}^k) \cup \{e^*\}$. Dacă $e^* \in E(T_k^*)$, atunci luăm $T_{k+1}^* = T_k^*$ și proprietatea are loc și pentru $k + 1$.

Să presupunem că $e^* \notin E(T_k^*)$. Atunci, $T_k^* + e^*$ are exact un circuit C , conținând $e^* = v_s v_{j^*}$. Deoarece $v_{j^*} \notin V(T_s^k)$, urmează că există o muchie $e_1 \neq e^*$ în C cu o extremitate în $V(T_s^k)$ și cealaltă în $V \setminus V(T_s^k)$. Din modul de alegere al e^* avem $c(e^*) \leq c(e_1)$ și $e_1 \in E(T_k^*) \setminus E(\mathcal{T}^k)$. Fie $T^1 = T_k^* + e^* - e_1$; evident, $T^1 \in \mathcal{T}_G$ (fiind conex cu $n - 1$ muchii). Deoarece $e_1 \in E(T_k^*) \setminus E(\mathcal{T}^k)$, avem $E(\mathcal{T}^{k+1}) \subseteq E(T^1)$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Demonstrație (continuare).



Pe de altă parte, deoarece $c(e^*) \leq c(e_1)$, avem $c(T^1) = c(T_k^*) + c(e^*) - c(e_1) \leq c(T_k^*)$.

Deoarece T_k^* este un MST al lui G , urmează că $c(T^1) = c(T_k^*)$, i. e., T^1 este un MST al lui G conținând toate muchiile din $E(\mathcal{T}^{k+1})$. Luând $T_{k+1}^* = T^1$ încheiem demonstrația teoremei. $\square$

Remarcă

- Demonstrația de mai sus rămâne adevărată și pentru funcții de cost $c : \mathcal{T}_G \rightarrow \mathbb{R}$ care satisfac: $\forall T \in \mathcal{T}_G, \forall e \in E(T), \forall e' \notin E(T)$

$$c(e') \leq c(e) \Rightarrow c(T + e' - e) \leq c(T).$$

- În metoda generală prezentată, modul de alegere a arborelui T_s^k nu este precizat în amănunt. Vom discuta două strategii foarte cunoscute.
- Prima strategie alege T_s^k ca fiind arborele de ordin maxim din familia $\mathcal{T}^k$.
- În cea de-a doua strategie T_s^k este unul dintre cei doi arbori din familia $\mathcal{T}^k$, legați printr-o muchie de cost minim printre toate muchiile cu extremități în arbori diferiți ai familiei.

Algoritmul lui Prim

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- În strategia lui Prim T_s^k este arborele de ordin maxim din familia $\mathcal{T}^k$.
- Urmează că la fiecare pas $k > 0$ al metodei generale, $\mathcal{T}^k$ are un arbore, $T_s^k = (V_s, E_s)$, cu $k + 1$ noduri și $n - k - 1$ arbori fiecare cu câte un nod.
- **Implementarea lui Dijkstra:** Fie α și β doi vectori de dimensiune n ; elementele lui α sunt noduri din $V(G)$ iar elementele lui β sunt numere reale, cu următoarea semnificație:

$$(S) \quad \forall j \in V \setminus V_s, \beta[j] = c(\alpha[j]j) = \min_{i \in V_s, ij \in E} c(ij)$$

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Algoritmul lui Prim

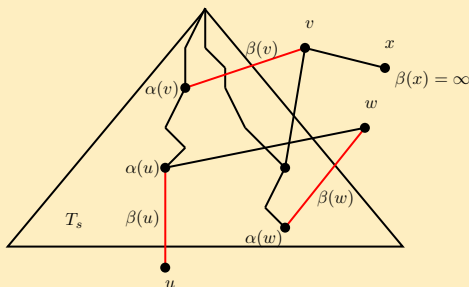
C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Algoritmul lui Prim

```
 $V_s \leftarrow \{s\}; E_s \leftarrow \emptyset; //$  pentru un  $s \in V$ .  
for ( $v \in V \setminus \{s\}$ ) do  
     $\alpha[v] \leftarrow s; \beta[v] \leftarrow c(sv); //$  dacă  $ij \notin E$ , atunci  $c(ij) = \infty$ .  
while ( $V_s \neq V$ ) do  
    find  $j^* \in V \setminus V_s$  a. î.  $\beta[j^*] = \min_{j \in V \setminus V_s} \beta[j];$   
     $V_s \leftarrow V_s \cup \{j^*\}; E_s \leftarrow E_s \cup \{\alpha[j^*]j^*\};$   
    for ( $j \in V \setminus V_s$ ) do  
        if ( $\beta[j] > c[j^*j]$ ) then  
             $\beta[j] \leftarrow c[j^*j]; \alpha[j] \leftarrow j^*;$ 
```

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Remarci



- Se observă că (S) este satisfăcută după inițializare. În bucla while, strategia metodei generale este respectată și de asemeni semnificația lui (S) este păstrată de testul din bucla for.
- **Complexitatea timp:** $\mathcal{O}(n - 1) + \mathcal{O}(n - 2) + \dots + \mathcal{O}(1) = \mathcal{O}(n^2)$ care este bună pentru grafuri de dimensiune $\mathcal{O}(n^2)$.

Algoritmul lui Kruskal

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- În algoritmul lui Kruskal T_s^k este unul dintre cei doi arbori din familia $\mathcal{T}^k$, legați printr-o muchie de cost minim printre toate muchiile cu extremități în arbori diferiți ai familiei.
- Această alegere poate fi făcută prin sortarea inițială a muchiilor crescător după cost și, după aceea, prin parcurgerea listei astfel obținute. Dacă notăm cu T arborele T_s^k , algoritmul poate fi descris astfel.

sort $E = \{e_1, e_2, \dots, e_m\}$ a. î. $c(e_1) \leq \dots \leq c(e_m)$;

$T \leftarrow \emptyset$; $i \leftarrow 1$;

while ($i \leq m$) do

if ($\langle T \cup \{e_i\} \rangle_G$ nu are circuite) then

$T \leftarrow T \cup \{e_i\}$;

$i++$;

- Sortarea poate fi făcută în $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$.
- Pentru implementarea eficientă a testului din bucla while, este necesar să reprezentăm mulțimile de noduri ale arbori, $V(T_1^k), V(T_2^k), \dots, V(T_{n-k}^k)$ (la fiecare pas k al metodei generale), și să testăm dacă muchia curentă are ambele extremități în aceeași mulțime.
- Aceste mulțimi vor fi reprezentate folosind arbori (care nu sunt, în general, subarbori ai grafului G). Fiecare astfel de arbore are o rădăcină care va fi folosită pentru a desemna mulțimea de noduri ale grafului G din acel arbore.
- Mai precis, avem a funcție $find(v)$ care determină cărei mulțimi îi aparține nodul v , adică, **returnează rădăcina arborelui care reține mulțimea lui v .**

Algoritmul lui Kruskal – Union-Find

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

- În metoda generală este necesară o reuniune (disjunctă) a mulțimilor de noduri a doi arbori (pentru a obține $\mathcal{T}^{k+1}$).
- Folosim procedura *union*(u, w) cu următoarea semnificație: realizează reuniunea a două mulțimi de noduri, una căreia îi aparține v și una căreia îi aparține w .
- Putem rescrie bucla while a algoritmului, folosind aceste două proceduri, după cum urmează:

```
while ( $i \leq m$ ) do
  let  $e_i = v w$ ;
  if ( $find(v) \neq find(w)$ ) then
    union( $v, w$ );
     $T \leftarrow T \cup \{e_i\}$ ;
   $i++$ ;
```

Algoritmul lui Kruskal – Union-Find – Prima soluție

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

- Tabloul $root[1..n]$ cu elemente din V are semnificația: $root[v] =$ rădăcina arborelui care reține mulțimea căreia îi aparține v .
- Adaugă la pasul de inițializare (corespunzând familiei $\mathcal{T}^0$):
for ($v \in V$) do
 $root[v] \leftarrow v$;
- Funcția *find* (cu complexitatea timp $\mathcal{O}(1)$):
function $find(v : V)$;
return $root[v]$;
- Procedura *union* (cu complexitatea timp $\mathcal{O}(n)$):
procedure $union(v, w : V)$;
for ($i \in V$) do
 if ($root[i] = root[v]$) then
 $root[i] = root[w]$;

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Algoritmul lui Kruskal – Union-Find – Prima soluție

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Analiza complexității timp:

- Sunt $\mathcal{O}(m)$ apeluri ale funcției *find* în timpul buclei *while*.
- În șirul de apeluri *find*, se intercalează exact $n-1$ apeluri *union* (un apel pentru fiecare muchie din MST-ul final).
- Astfel, time necesar buclei *while* este $\mathcal{O}(m\mathcal{O}(1) + (n-1)\mathcal{O}(n)) = \mathcal{O}(n^2)$.

Complexitatea timp a algoritmului este $\mathcal{O}(\max(m \log n, n^2))$.

Dacă G are multe muchii, $m = \mathcal{O}(n^2)$, algoritmul lui Prim este mai eficient.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

- Tabloul $pred[1..n]$ cu elemente din $V \cup \{0\}$ are semnificația $pred[v] =$ nodul dinaintea lui v pe drumul unic către v de la rădăcina arborelui care reține mulțimea căruia îi aparține v .
- Aduagă la pasul de inițializare (corespunzând familiei $\mathcal{T}^0$):
for ($v \in V$) do
 $pred[v] \leftarrow 0$;
- Funcția $find(v)$ are complexitatea în $\mathcal{O}(h(v))$, unde $h(v)$ este lungimea drumului din arbore de la v la rădăcina acestui arbore:
function $find(v : V)$;
 $i \leftarrow v$; // o variabilă locală.
 while ($pred[i] > 0$) do
 $i \leftarrow pred[i]$;
 return i ;

Algoritmul lui Kruskal – Union-Find – A doua soluție

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- Procedura **union** (de complexitate timp $\mathcal{O}(1)$) este apelată doar pentru noduri rădăcină:

```
procedure union(root1, root2 : V)  
  pred[root1]  $\leftarrow$  root2;
```

- Bucla **while** a algoritmului este modificată astfel:

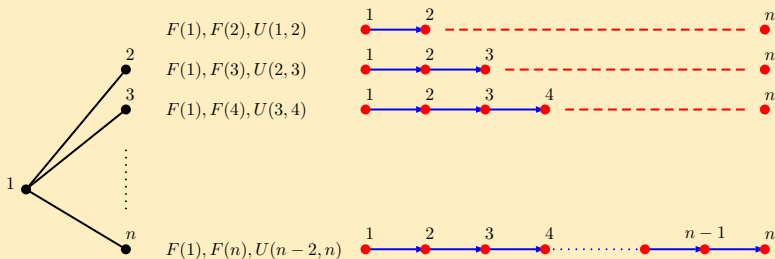
```
while (i  $\leq$  m) do  
  let ei = vw; x  $\leftarrow$  find(v); y  $\leftarrow$  find(w);  
  if (x  $\neq$  y) then  
    union(x, y);  
    T  $\leftarrow$  T  $\cup$  {ei};  
  i ++;
```

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Algoritmul lui Kruskal – Union-Find – A doua soluție

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Dacă executăm bucla **while** în această formă pentru graful $G = K_{1,n-1}$ cu lista sortată amuchiilor, $E = \{12, 13, \dots, 1n\}$, atunci șirul de apeluri ale celor două proceduri este (F și U abreviază **find** și **union**):



Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Algoritmul lui Kruskal – Union-Find – A doua soluție

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

- Astfel, această formă a algoritmului are complexitatea timp $\Omega(n^2)$ (chiar dacă graful este rar).
- Neajunsul acestei implementări este dat de faptul că, în procedura **union**, rădăcină a noului arbore devine rădăcina acelui arbore care reține un număr mai mic de noduri, ceea ce implică o mărire a lui $h(v)$ la $\mathcal{O}(n)$ în timpul algoritmului.
- Putem evita acest neajuns ținând în rădăcina fiecărui arbore cardinalul mulțimii pe care arborele o reține. Mai precis, semnificația $pred[v]$, unde v este o rădăcină, este:

$pred[v] < 0 \Leftrightarrow v$ este rădăcina unui arbore

care reține o mulțime cu $-pred[v]$ noduri

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Algoritmul lui Kruskal – Union-Find – A doua soluție

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- Pasul de inițializare

```
for ( $v \in V$ ) do  
     $pred[v] \leftarrow -1$ ;
```

- Procedura *union* are complexitatea $\mathcal{O}(1)$ pentru a întreține noua semnificație:

```
procedure union( $root_1, root_2 : V$ )  
     $t \leftarrow pred[root_1] + pred[root_2]$ ;  
    if ( $-pred[root_1] \geq -pred[root_2]$ ) then  
         $pred[root_2] \leftarrow root_1$ ;  $pred[root_1] \leftarrow t$ ;  
    else  
         $pred[root_1] \leftarrow root_2$ ;  $pred[root_2] \leftarrow t$ ;
```

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Algoritmul lui Kruskal – Union-Find – A doua soluție

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Afirmație. Cu această implementare a procedurilor `find` și `union` algoritmul are următorul invariant:

$$(*) \forall v \in V, -pred[find(v)] \geq 2^{h(v)}$$

Cu alte cuvinte, numărul de noduri din arborele căruia îi aparține v este cel puțin 2 la puterea "distanța de la v la rădăcină".

Demonstrația afirmației. După pasul de inițializare avem $h(v) = 0$, $find(v) = v$, și $-pred[v] = 1$, $\forall v \in V$, deci $(*)$ are loc cu egalitate.

Să presupunem că $(*)$ are loc înaintea unei iterații din bucla `while`. Sunt posibile două cazuri:

- În această iterație `while` nu este apelată `union`. Tabloul `pred` nu este actualizat, deci $(*)$ are loc și după această iterație.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Algoritmul lui Kruskal – Union-Find – A doua soluție

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- În această iterație **while** avem un apel al procedurii **union**. Fie $union(x, y)$ acest apel, și să presupunem că în procedura **union** se execută atribuirea $pred[y] \leftarrow x$. Aceasta înseamnă că înaintea acestei iterații avem $-pred[x] \geq -pred[y]$.

Nodurile v pentru care $h(v)$ se modifică în această iterația sunt acelea pentru care, înaintea iterației aveam $find(v) = y$ și $-pred[y] \geq 2^{h(v)}$.

După iterația **while**, avem $h'(v) = h(v) + 1$ și $find'(v) = x$. Astfel, trebuie să verificăm că $-pred'[x] \geq 2^{h'(v)}$. Într-adevăr, $-pred'[x] = -pred[x] - pred[y] \geq 2 \cdot (-pred[y]) \geq 2 \cdot 2^{h(v)} = 2^{h(v)+1} = 2^{h'(v)}$.

Urmează că (*) este un invariant al algoritmului. $\square$

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Algoritmul lui Kruskal – Union-Find – A doua soluție

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Aplicând logaritmul în (*) obținem

$$h(v) \leq \log(-\text{pred}[\text{find}(v)]) \leq \log n, \forall v \in V.$$

Complexitatea timp a buclei **while** este

$$\mathcal{O}(n - 1 + 2m \log n) = \mathcal{O}(m \log n).$$

Astfel, această a doua implementare a procedurilor **union–find** dă o complexitate timp a algoritmului Kruskal de $\mathcal{O}(m \log n)$.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Complexitatea timp a buclei **while** din soluția de mai sus este datorată șirului de apeluri **find**. **Tarjan (1976)** a observat că un apel cu $h(v) > 1$ poate să "colapseze" drumul din arbore de la v la rădăcină, fără a modifica timpul $\mathcal{O}(h(v))$, făcând $h(x) = 1$ pentru toate nodurile x de pe drum. În acest fel, viitoarele apeluri **find** pentru aceste noduri vor lua mai puțin timp. Mai precis, funcția **find** devine:

```
function find( $v : V$ ); //  $i, j, aux$  sunt variabile locale.
```

```
 $i \leftarrow v$ ;
```

```
while ( $pred[i] > 0$ ) do
```

```
     $i \leftarrow pred[i]$ ;
```

```
 $j \leftarrow v$ ;
```

```
while ( $pred[j] > 0$ ) do
```

```
     $aux \leftarrow pred[j]$ ;  $pred[j] \leftarrow i$ ;  $j \leftarrow aux$ ;
```

```
return  $i$ ;
```

Dacă $A : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ este funcția lui Ackermann dată prin:

$$(1) \quad A(m, n) = \begin{cases} n + 1, & \text{dacă } m = 0 \\ A(m - 1, 1), & \text{dacă } m > 0 \text{ și } n = 0 \\ A(m - 1, A(m, n - 1)), & \text{dacă } m > 0 \text{ și } n > 0 \end{cases}$$

și dacă notăm, $\forall m \geq n > 0$,

$$\alpha(m, n) = \min \{z : A(z, 4 \lceil m/n \rceil) \geq \log n, z \geq 1\}$$

obținem că **complexitatea timp a buclei while folosind union din cea de-a doua soluție și fiind de mai sus, devine $\mathcal{O}(m \cdot \alpha(m, n))$.**

Să notăm că $\alpha(m, n)$ este o funcție care crește foarte încet (pentru valori practice ale lui n , $\alpha(m, n) \leq 3$); astfel cea de-a treia soluție este practic o implementare liniară ($\mathcal{O}(m)$) a algoritmului lui Kruskal.

Fie $G = (V, E)$ un (multi)graf. Dacă $A \subseteq E$ și $v \in V$, notăm $d_A(v) = |\{e : e \in A, e \text{ incidentă cu } v\}|$, i. e., gradul lui v în subgraful generat de A , $\langle A \rangle_G$.

Definiție

Un cuplaj (mulțime independentă de muchii) în G este o mulțime de muchii $M \subseteq E$ astfel încât

$$d_M(v) \leq 1, \forall v \in V.$$

Familia tuturor cuplajelor din graful G este notată cu $\mathcal{M}_G$:

$$\mathcal{M}_G = \{M : M \subseteq E, M \text{ cuplaj în } G\}.$$

Se observă că $\mathcal{M}_G$ satisface:

- (i) $\emptyset \in \mathcal{M}_G$;
- (ii) $M \in \mathcal{M}_G, M' \subseteq M \Rightarrow M' \in \mathcal{M}_G$.

Fie $M \in \mathcal{M}_G$ un cuplaj.

- Un nod $v \in V$ cu $d_M(v) = 1$ este numit **saturat** de către M , iar mulțimea tuturor nodurilor lui G saturate de M este notată cu $S(M)$. Evident,

$$S(M) = \bigcup_{e \in M} e, \text{ și } |S(M)| = 2 \cdot |M|.$$

- Un nod $v \in V$ cu $d_M(v) = 0$ este numit **expus** (relativ) la M , iar mulțimea tuturor nodurilor lui G expuse relativ la M este notată cu $E(M)$. Evident că $E(M) = V \setminus S(M)$, și $|E(M)| = |V| - 2 \cdot |M|$.

Definiție

O acoperire cu muchii a lui G este o mulțime de muchii $F \subseteq E$ astfel încât

$$d_F(v) \geq 1, \forall v \in V(G).$$

Familia acoperirilor cu muchii ale grafului G este notată cu $\mathcal{F}_G$:

$$\mathcal{F}_G = \{F : F \subseteq E, F \text{ acoperire cu muchii a lui } G\}.$$

$\mathcal{F}_G$ are următoarele proprietăți

- (i) $\mathcal{F}_G \neq \emptyset \Leftrightarrow G$ nu are noduri izolate (caz în care $E \in \mathcal{F}_G$);
- (ii) $F \in \mathcal{F}_G, F' \supseteq F \Rightarrow F' \in \mathcal{F}_G$.

Problem acoperirii minime cu muchii:

P₂ dat un graf $G = (V, E)$, găsiți $F^* \in \mathcal{F}_G$ astfel încât

$$|F^*| = \min_{F \in \mathcal{F}_G} |F|.$$

Teorema 2

(Norman-Rabin, 1959) Fie $G = (V, E)$ un graf de ordin n , fără noduri izolate. Dacă M^* este un cuplaj de cardinal maxim în G și F^* este o acoperire minimă cu muchii a lui G , atunci

$$|M^*| + |F^*| = n.$$

Demonstrație: " $\leq$ " Fie M^* un cuplaj de cardinal maxim în G ; considerăm următorul algoritm:

```
 $F \leftarrow M^*;$ 
for ( $v \in E(M^*)$ ) do
  find  $v' \in S(M^*)$  a. î.  $vv' \in E;$ 
   $F \leftarrow F \cup \{vv'\};$ 
```


Demonstrație (continuare).

Să notăm că, $\forall v \in E(M^*)$, deoarece G nu are noduri izolate, există o muchie incidentă cu v , și, cum M^* este maximal relativ la relația de incluziune, această muchie are celălalt capăt în $S(M^*)$.

Mulțimea F de muchii construită este o acoperire cu muchii și $|F| = |M^*| + |E(M^*)| = |M^*| + n - 2 \cdot |M^*| = n - |M^*|$. Astfel

$$(2) \qquad |F^*| \leq |F| = n - |M^*|.$$

" $\geq$ " Fie F^* o acoperire minimă cu muchii a lui G ; considerăm următorul algoritm:

```
 $M \leftarrow F^*;$   
for ( $\exists v \in V : d_M(v) > 1$ ) do  
  find  $e \in M$  incidentă cu  $v$ ;  
   $M \leftarrow M \setminus \{e\};$ 
```

Demonstrație (continua).

Evident că algoritmul construiește un cuplaj M în G . Dacă muchia e incidentă cu v (eliminată din M într-o iterație **while**) este $e = vv'$, atunci $d_M(v') = 1$ și în următoarea iterație vom avea $d_M(v') = 0$, astfel la fiecare iterație **while** este creat un nod expus relativ la cuplajul final, M (dacă ar exista o altă muchie e' , în mulțimea curentă M , incidentă cu v' , atunci deoarece $e \in F^*$, $F^* \setminus \{e\}$ ar fi o acoperire cu muchii, în contradicție cu alegerea lui F^*).

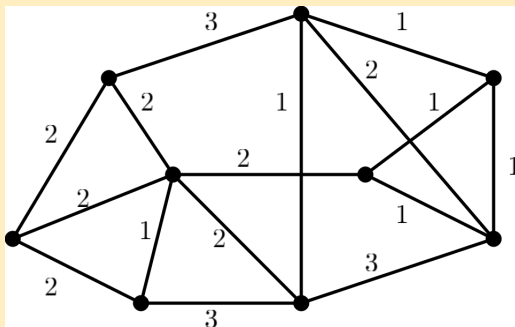
Astfel, dacă M este cuplajul construit de algoritm, avem: $|F^*| - |M| = |E(M)| = n - 2 \cdot |M|$, i. e.,

$$(3) \quad |F^*| = n - |M| \geq n - |M^*|.$$

Din (2) și (3) derivă concluzia teoremei. $\square$

Exerciții pentru seminarul din săptămâna următoare

Exercițiul 1'. Determinați un arbore parțial de cost minim în graful de mai jos.



Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 1. Fie $G = (V, E)$ un graf conex și $c : E \rightarrow \mathbb{R}$ o funcție de cost pe muchiile sale. O submulțime $A \subseteq E$ este numită *tăietură* dacă există o bipartiție (S, T) a lui V astfel încât $A = \{uv \in E : u \in S, v \in T\}$ ($G \setminus A$ nu mai este conex).

- (a) Dacă în orice tăietură există o singură muchie de cost minim, atunci G conține un singur arbore parțial de cost minim.
- (b) Arătați că, dacă c este funcție injectivă, atunci G conține un singur arbore parțial de cost minim.
- (c) Reciprocele afirmațiilor de mai sus sunt adevărate?

Exercițiul 2. Fie $G = (V, E)$ un graf conex de ordin n , $c : E \rightarrow \mathbb{R}$, și $\mathcal{T}_G^{\min}$ familia arborilor săi parțiali de cost (c) minim. Definim $H = (\mathcal{T}_G^{\min}, E(H))$ unde $T_1 T_2 \in E(H) \iff |E(T_1) \Delta E(T_2)| = 2$. Arătați că H este conex și că diametrul său este cel mult $n - 1$.

Exerciții pentru seminarul din săptămâna următoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 3. Fie $G = (V, E)$ un graf conex și $c : E \rightarrow \mathbb{R}$. Pentru un arbore parțial $T = (V, E') \in \mathcal{T}_G$, și $v \neq w \in V$ notăm cu P_{vw}^T singurul vw -drum din T . Arătați că un arbore parțial $T^* = (V, E^*)$ este de cost minim dacă și numai dacă

$$\forall e = vw \in E \setminus E^*, \forall e' \in E(P_{vw}^{T^*}), \text{ avem } c(e) \geq c(e').$$

- Graph Algorithms - C. Croitoru - Graph Algorithms - C. Croitoru - Graph Algorithms - C.
G. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 4. Fie $G = (V, E)$ un graf 2-muchie-conex și $c : E \rightarrow \mathbb{R}$. Dacă $T = (V, E')$ este arbore parțial de cost minim al lui G și $e \in E'$, $T - e$ are exact două componente conex T'_1 și T'_2 , respectiv. Notăm cu $e_T \neq e$ o muchie de cost minim în tăietura generată de $(V(T'_1), V(T'_2))$ în $G - e$. Arătați că, dacă T^* este un arbore parțial de cost minim al lui G , și $e \in E(T^*)$, atunci $T^* - e + e_{T^*}$ este un arbore parțial de cost minim $G - e$.

Exercițiul 5. Fie $G = (V, E)$ un graf conex și $c : E \rightarrow \mathbb{R}$ o funcție de cost injectivă pe muchiile sale. Considerăm următorul algoritm

```
for ( $e \in E$ ) do
     $\gamma(e) \leftarrow r$ ; // toate muchiile sunt colorate cu roșu; în timpul execuției
    vor fi roșii, albastre sau verzi
while (( $\exists A \subseteq E$ , tăietură cu  $\gamma(e') \neq v$ , unde  $c(e') = \min_{e \in A} c(e)$ ) sau
    ( $\exists C$ , un circuit cu  $\gamma(e') \neq a$ , unde  $c(e') = \max_{e \in C} c(e)$ )) do
    pentru o tăietură,  $A$ ,  $\gamma(e') \leftarrow v$ ;
    pentru un circuit,  $C$ ,  $\gamma(e') \leftarrow a$ ;
return  $H = (V, \{e \in E : \gamma(e) = v\})$ ;
```

Arătați că

- (a) o muchie aparține unui arbore parțial de cost minim dacă și numai dacă este de cost minim într-o tăietură;
- (b) o muchie nu aparține nici unui arbore parțial de cost minim dacă și numai dacă este de cost maxim într-un circuit;

Exercițiul 5 (continuare).

- (c) algoritmul nu se oprește câtă vreme mai există muchii roșii în graf;
- (d) algoritmul se oprește pentru orice alegere a muchiilor e' , iar H este singurul arbore parțial de cost minim din G .

Exercițiul 6. Fie H un graf conex, $\emptyset \neq A \subseteq V(H)$, și $w : E(H) \rightarrow \mathbb{R}_+$. Un arbore Steiner pentru (H, A, w) este un arbore $T(H, A, w) = (V_T, E_T) \subseteq H$ cu $A \subseteq V_T$ care are costul minim printre toți arborii care conțin A , și care sunt subgrafuri ale lui H :

$$\begin{aligned} s[T(H, A, w)] &= \sum_{e \in E_T} w(e) = \\ &= \min \left\{ \sum_{e \in E_{T'}} w(e) : T' = (V_{T'}, E_{T'}) \text{ arbore în } H, A \subseteq V_{T'} \right\} \end{aligned}$$

- (a) Arătați că un arbore Steiner poate fi determinat în timp polinomial dacă $A = V(H)$ sau $|A| = 2$.

Exercițiul 6 (continuare).

- (b) Fie $G = (V, E)$ un graf conex cu $V = \{1, 2, \dots, n\}$, și $A \subseteq V$; avem de asemenea și o funcție de cost $c : E \rightarrow \mathbb{R}_+$. Considerăm graful complet K_n (cu $V(K_n) = V$) și definim $\bar{c} : E(K_n) \rightarrow \mathbb{R}_+$:

$$\bar{c}(ij) = \min \left\{ c(P) = \sum_{e \in E(P)} c(e) : P \text{ este } ij\text{-drum în } G \right\}$$

Demonstrați că $s[T(G, A, c)] = s[T(K_n, A, \bar{c})]$ și arătați cum se poate construi un arbore Steiner $T(K_n, A, \bar{c})$ dintr-un arbore Steiner $T(G, A, c)$.

- (c) Arătați că există un arbore Steiner $T(K_n, A, \bar{c})$ astfel încât toate nodurile sale din afara lui A au gradul cel puțin 3. Folosind această proprietate arătați că există un arbore Steiner $T(K_n, A, \bar{c})$ cu cel mult $2|A| - 2$ noduri.

Exercițiul 7. Considerăm o ordonare $E = \{e_1, e_2, \dots, e_m\}$ a muchiilor unui graf conex $G = (V, E)$ de ordin n . Pentru orice submulțime $A \subseteq E$ definim $x^A \in GF^m$ vectorul caracteristic m -dimensional al mulțimii A : $x_i^A = 1 \Leftrightarrow e_i \in A$. GF^m este spațiul m -dimensional peste $\mathbb{Z}_2$.

- (a) Arătați că submulțimea vectorilor caracteristici corespunzători tuturor tăieturilor din G împreună cu vectorul nul este un subspațiu X al lui GF^m .
- (b) Arătați că submulțimea vectorilor caracteristici corespunzători tuturor circuitelor din G generează un subspațiu U al lui GF^m care este ortogonal pe X .
- (c) Arătați că $\dim(X) \geq n - 1$.
- (d) Arătați că $\dim(U) \geq m - n + 1$.
- (e) În final, dovediți că inegalitățile de mai sus sunt de fapt egalități.

Exerciții pentru seminarul din săptămâna următoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 8. Fie $G = (V, E)$ un graf conex și $c : E \rightarrow \mathbb{R}$ o funcție de cost pe muchiile sale.

a) Fie T^* un arbore parțial de cost c -minim al lui G și $\epsilon > 0$. Arătați că T^* este singurul arbore parțial de cost $\bar{c}$ -minim al lui G , unde

$$\bar{c}(e) = \begin{cases} c(e) - \epsilon, & \text{dacă } e \in E(T^*) \\ c(e), & \text{altfel} \end{cases}$$

b) Deduceți de aici că pentru orice arbore parțial de cost minim, T^* , al lui G există o ordonare a muchiilor lui G astfel încât algoritmul lui Kruskal returnează T^* .

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul din săptămâna următoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

Exercițiul 9. Fie $G = (V, E)$ un graf conex și $c : E \rightarrow \mathbb{R}$ o funcție de cost injectivă. Fie T^* un arbore parțial de cost minim al lui G și T_0 un arbore parțial cu cel de-al doilea cel mai mic cost în G .

- (a) T_0 este unicul arbore parțial cu cel de-al doilea cel mai mic cost în G ?
- (b) Arătați că $|E(T^*) \Delta E(T_0)| = 2$.
- (c) Descrieți un algoritm pentru a determina un arbore parțial cu cel de-al doilea cel mai mic cost în G .

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul din săptămâna următoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

Exercițiul 10. Fie $G = (V, E)$ un graf conex și $c : E \rightarrow \mathbb{R}$ o funcție de cost. Adevărat sau fals? (Justificați răspunsurile!)

- (a) Orice muchie de cost minim din G este conținută într-un anume arbore parțial de cost minim din G .
- (b) Dacă G are un circuit, C , cu o singură muchie de cost minim, atunci acea muchie este conținută în orice arbore parțial de cost minim din G .
- (c) Dacă o muchie este conținută într-un arbore parțial de cost c minim din G , atunci acea muchie este de cost minim într-o anumită tăietură a lui G .

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții pentru seminarul din săptămâna următoare

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 11. Determinați numărul de cuplaje maxime ale următorului graf:



C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

Exercițiul 12. Doi copii se joacă pe un graf dat, G , astfel: fiecare alege alternativ un nod nou $v_0, v_1, \dots$ așa încât, pentru orice $i > 0$, v_i este adiacent cu v_{i-1} . Jucătorul care nu mai poate alege un nod nou pierde jocul. Arătați că jucătorul care începe jocul are întotdeauna o strategie de câștig dacă și numai dacă G nu are un cuplaj perfect.

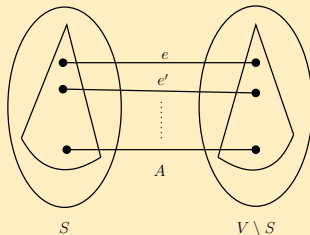
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 1. Soluție.

- (a) Ștergând o muchie e dintr-un arbore parțial T al lui G , obținem exact două componente conexe (subarbori ai lui T - **de ce?**): unul cu mulțimea nodurilor S iar celălalt cu $V \setminus S$; e aparține tăieturii generate de bipartiția($S, V \setminus S$):



Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms

- Dacă T este un arbore parțial de cost minim, atunci e este muchia de cost minim în tăietură corespunzătoare de mai sus (de ce?).
- Presupunem prin reducere la absurd că există doi arbori parțiali de cost minim $T_1 = (V, E_1) \neq T_2 = (V, E_2)$. Fie $e_1 \in E_1 \setminus E_2$.
- $T_1 - e_1$ are două componente conexe cu mulțimile de noduri S și $V \setminus S$.
- În tăietura A generată de $(S, V \setminus S)$, e_1 este muchia de cost minim.
- Pe de altă parte $T_2 + e_1$ conține doar un circuit C (de ce?).
- C intersectează tăietura în cel puțin o altă muchie (de ce?), să zicem $e_2 \in E_2 \setminus E_1$.
- $T_2 + e_1 - e_2$ este a arbore parțial (de ce?) cu $c(T_2 + e_1 - e_2) < c(T_2)$ - contradicție.

- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

(b) and (c) are consequences of (a). **De ce?**

Exercițiul 4. Soluție. Fie T_0 un arbore parțial de cost minim în $G - e$.

- $c(T_0) \leq c(T^* - e + e_{T^*})$ (**de ce?**).
- $T_0 + e$ conține exact un circuit C care intersectează tăietură generată de $(V(T'_1), V(T'_2))$ în cel puțin o altă muchie $e_0 \neq e$ (**de ce?**).
- $c(e_0) \geq c(e_{T^*})$ (**de ce?**).
- $T_0 + e - e_0 \in \mathcal{T}_G$, deci $c(T_0 + e - e_0) \geq c(T^*)$ și

$$c(T^* - e + e_{T^*}) \geq c(T_0) \geq c(T^*) - c(e) + c(e_0) \geq c(T^*) - c(e) + c(e_{T^*})$$

- Astfel, inegalitățile de mai sus devin egalități iar $(T^* - e + e_{T^*})$ trebuie să fie arbore parțial de cost minim în $G - e$.

Exercițiul 5. Soluție. Fie $T^* = (V, E^*)$ un arbore parțial de cost minim în G .

(a) " $\Rightarrow$ " Fie $e \in E^*$; $T^* - e$ este o pădure cu doi subarbori T_1^* and T_2^* .

- În tăietură generată de $V(T_1^*)$ și $V(T_2^*)$ considerăm o muchie de cost minim e' .
- Presupunem prin reducere că $e' \neq e$, atunci $c(e) > c(e')$, dar $T = T^* - e + e' \in \mathcal{T}_G$ și $c(T) < C(T^*)$ (de ce?) - contradicție.

" $\Leftarrow$ " Fie A o tăietură și $e' \in A$ cu $c(e') = \min_{e \in A} c(e)$.

- Presupunem prin reducere că $e' \notin E^*$.
- Atunci $T^* + e'$ conține un circuit C și fie $e'' \in E(C) \cap A$ o muchie diferită de e' (de ce există o astfel de muchie?).
- $c(e') < c(e'')$ și $T^* + e' - e'' \in \mathcal{T}_G$ cu $c(T^* + e' - e'') < c(T^*)$ (de ce?) - contradicție.

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

(b) " $\Rightarrow$ " Dacă $e \notin E^*$, atunci pe circuitul C din $T^* + e'$, e' este de cost maxim. De ce?

" $\Leftarrow$ " Dacă e' este de cost maxim pe circuitul C iar $e' \in E^*$, atunci $T^* - e'$ este o pădure cu doi subarbori T_1^* și T_2^* .

- În tăietura generată de $V(T_1^*)$ și $V(T_2^*)$ C trebuie să conțină și o altă muchie e'' (de ce?).
- Cum $c(e') > c(e'')$ obținem $T = T^* - e' + e'' \in \mathcal{T}_G$ și $c(T) < c(T^*)$ - contradicție.

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

(c) Fie $e = uv$ o muchie roșie, definim

$$P = \{x \in V : \text{există un drum verde de la } u \text{ la } x \text{ în } G\}.$$

- Dacă $v \in P$, atunci $C = P_{uv} + e$ este un circuit pe care e este de cost maxim (de ce?) și va fi colorată în albastru de către algoritm.
- Dacă $v \notin P$, atunci tăietura generată de P și $V \setminus P$ nu conține muchii verzi (de ce?), deci algoritmul va colora cu verde o muchie de pe el.

(d) Muchiile nu se recolorează (de ce?), deci bucla **while** se termină după cel mult $|E|$ iterații când nu vor mai fi muchii roșii.

- At the end of the algorithm we have only blue and green muchii, the blue ones doesn't belong to T^* , hence all the blue muchii are in T^* .
- Ca rezultat secundar: T^* este singurul arbore parțial de cost minim în G (de ce?).

Exercițiul 7. Soluție.

- (a) Trebuie demonstrat că pentru orice două tăieturi A_1, A_2 and $\alpha_1, \alpha_2 \in \{0, 1\}$, avem $\alpha_1 x^{A_1} + \alpha_2 x^{A_2} = x^A$, unde A este o tăietură sau mulțimea vidă.
- Evident că $A = \emptyset \Leftrightarrow A_1 = A_2$ (de ce?).
 - Deoarece $\alpha_1, \alpha_2 \in \mathbb{Z}_2$, va fi suficient să arătăm că, dacă $A_i = \{uv \in E : u \in S_i, v \in T_i\}$, $i = 1, 2$, sunt tăieturi distincte, atunci $x^{A_1} + x^{A_2} = x^A$, unde A este o tăietură în G ((S_i, T_i) sunt bipartiții ale lui V).
 - Este ușor de văzut că $A = A_1 \Delta A_2$ (de ce?).
 - Se poate demonstra că A este tăietura $\{uv \in E : u \in S, v \in T\}$, unde $S = S_1 \Delta S_2$, $T = V \setminus S$. Cum?

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru

(b) Deoarece $U = \text{span}(\{x^{E(C)} : C \text{ circuit în } G\})$, este suficient să arătăm că pentru orice circuit C și orice tăietură A , $x^A \perp x^{E(C)}$.

- Dacă alegem o tăietură A și un circuit C , este ușor de văzut că $|A \cap E(C)| \equiv 0 \pmod{2}$ (de ce?).
- Astfel,

$$\langle x^A, x^{E(C)} \rangle \equiv \sum_{i=1}^m x_i^A x_i^{E(C)} \stackrel{\text{de ce?}}{=} |A \cap E(C)| \equiv 0 \pmod{2}.$$

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

(c) Vom pune în evidență $(n - 1)$ vectori linear independenți din X .

- Pentru orice $v \in V$, let $A_v = \{vw \in E : w \neq v\} \neq \emptyset$.
- Dacă $V = \{v_1, v_2, \dots, v_n\}$, atunci $x^{A_1}, x^{A_2}, \dots, x^{A_{n-1}}$ sunt independenți. Fie $\{i_1, i_2, \dots, i_k\} \subseteq \{1, 2, \dots, n - 1\}$.
- Cum G este conex, există o muchie $e_h = v_{i_j} v_q \in E$, unde $q \notin \{i_1, i_2, \dots, i_k\}$ (de ce?).
- Avem

$$\left(\sum_{j=1}^k x^{A_{i_j}} \right)_h = 1, \text{ thus } \sum_{j=1}^k x^{A_{i_j}} \neq 0 \text{ (de ce?).}$$

- O bază în X va avea cel puțin $(n - 1)$ vectori, deci $\dim(X) \geq n - 1$.

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

(d) Vom pune în evidență $(m - n + 1)$ vectori liniar independenți în U .

- Fie T un arbore parțial al lui G ; pentru orice $e \in E \setminus E(T)$, $T + e$ conține exact un circuit C_e .
- Fie $\{e_{i_1}, e_{i_2}, \dots, e_{i_p}\} \subseteq E \setminus E(T)$.
- Evident că $e_{i_1} \notin E(C_{e_{i_j}})$, pentru $2 \leq j \leq p$ deci

$$\left(\sum_{j=1}^p x^{E(C_{e_{i_j}})} \right)_{i_1} = 1, \text{ thus } \sum_{j=1}^p x^{E(C_{e_{i_j}})} \neq 0 \text{ (de ce?).}$$

- Astfel $\dim(U) \geq m - n + 1$.

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

(e) Am arătat că

$$\dim(X) + \dim(U) \geq m = \dim(GF^m)$$

- Mai știm că $U \subseteq X^\perp$ (de ce?).
- Deci $\dim(U) \leq \dim(X^\perp) = \dim(GF^m) - \dim(X) \geq \dim(U)$.
- Adică $\dim(X) + \dim(U) = m$ - de unde se obțin egalitățile dorite.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph

Exercițiul 9. Soluție. Dacă c este injectivă, există un singur arbore parțial de cost minim în G (vezi ex. 1).

- (a) Nu. $G \equiv K_4$, $V(G) = \{x, y, z, t\}$; $c(xy) = 2$, $c(yz) = 4$, $c(zt) = 3$,
 $c(tx) = 1$, $c(xz) = 5$, și $c(yt) = 6 \dots$

Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

(b) Evident că $p = |E(T^*) \setminus E(T_0)| = |E(T_0) \setminus E(T^*)| = |E(T^*) \Delta E(T_0)|/2$.

- Presupunem prin reducere la absurd că $k \geq 2$ și fie e_1 o muchie de cost minim din $E(T^*) \setminus E(T_0)$.
- $T_0 + e_1$ conține un circuit C și există o muchie $e_0 \in E(C) \cap (E(T_0) \setminus E(T^*))$.
- Vom arăta că $c(e_0) > c(e_1)$. Altfel $T^* + e_0$ conține un circuit C' și există o muchie $e_2 \in E(C') \cap (E(T^*) \setminus E(T_0))$.
- $T' = T^* + e_0 - e_2 \in \mathcal{T}_G$, deci $c(T') < c(T^*)$ și $c(e_2) < c(e_0) < c(e_1)$ - contradicție (de ce?).

Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru
- Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms
* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -

- Astfel, $c(e_0) > c(e_1)$. Arborele $T'' = T_0 + e_1 - e_0 \in \mathcal{T}_G$ are costul $c(T'') < c(T_0)$.
 - Urmează că T'' este arbore parțial de cost minim, dar $T'' \neq T^*$ - contradicție (de ce?).
- (c) Aplicăm un algoritm pentru determinarea unui arbore parțial de cost minim pentru orice graf $G - e$, $\forall e \in E(T^*)$ și păstrăm arborele parțial de cost minim obținut.
- Cât este complexitatea timp?
 - Remarcă: o parte dintre grafurile de mai sus pot să nu fie conexe.

* C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph
Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru -
Graph Algorithms * C. Croitoru - Graph Algorithms * C. Croitoru - Graph Algorithms *

Exerciții rezolvate (parțial)

Exercițiul 10. Soluție.

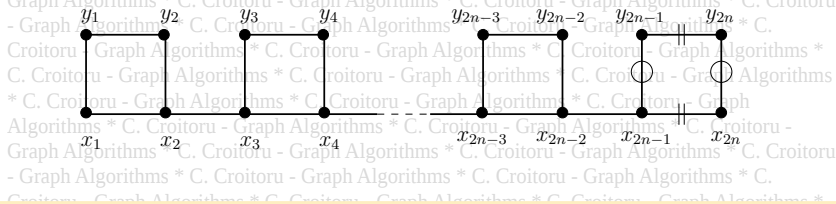
- (a) Adevărat, folosind algoritmul lui Kruskal (cum?). Adevărat (de ce?).
- (b) Fals (de ce?).
- (c) Adevărat (de ce?).

[illegible]

Exerciții rezolvate (parțial)

Exercițiul 11. Soluție.

- Fie $\#_{\max}(G_n)$ numărul de cuplaje maxime din G_n .
- Se poate folosi inducția după n pentru a arăta că numărul de cuplaje perfecte din graful de mai sus este 2^n .



Exercițiul 13. Soluție. Fie $G = (S, T; E)$ următorul graf bipartit: $T = \mathcal{A}$, $S = \mathcal{B}$, și $A_i B_j \in E$ dacă $A_i \cap B_j \neq \emptyset$.

- Se folosește teorema lui Hall (cum?).